



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش نهم

پیاده‌سازی حافظه‌ی شرکت‌پذیر سه‌گانه (TCAM)

خلاصه

در این آزمایش یک پیمانه‌ی TCAM به اندازه‌ی ۱۶ ثبات ۱۶ بیتی طراحی و پیاده‌سازی شده است. حافظه‌ی معمولی با آدرس کار می‌کند: آدرس می‌دهی و محتوا می‌گیری. حافظه‌ی شرکت‌پذیر (CAM) برعکس عمل می‌کند: محتوا می‌دهی و آدرس می‌گیری، و این کار را با مقایسه‌ی هم‌زمان همه‌ی خانه‌ها انجام می‌دهد. TCAM یک قدم جلوتر می‌رود و اجازه می‌دهد در هر خانه، علاوه بر ۰ و ۱، نماد سوم X هم ذخیره شود که یعنی «این جایگاه مقایسه نشود».

نماد سوم چطور ذخیره می‌شود

سخت‌افزار سطح منطقی سومی برای ذخیره کردن ندارد، پس هر بیت با دو فلیپ‌فلاپ نگه داشته می‌شود:

نماد ذخیره‌شده	value	care
۰	۰	۱
۱	۱	۱
X (مقدار بی‌اهمیت است)	—	۰

و یک جایگاه وقتی با بیت جست‌وجو موافق است که:

$$\text{agree} = \overline{\text{care}} + \overline{(\text{value} \oplus \text{search})}$$

یعنی یا اصلاً قرار نیست مقایسه شود، یا مقایسه شده و برابر درآمده است. کل سلول سه‌گانه همین است:

```

1 module tcam_cell (
2   input wire Clk,
3   input wire RstN,
4   input wire we,
5   input wire d,
6   input wire care,
7   input wire s,
8   output wire agree,
9   output wire d_q,
10  output wire care_q
11 );
12  reg vq, cq;
13
14  always @(posedge Clk or negedge RstN) begin
15    if (!RstN) begin
16      vq <= 1'b0;
17      cq <= 1'b0;
18    end else if (we) begin
19      vq <= d;
20      cq <= care;
21    end
22  end
23
24  assign agree = ~cq | ~(vq ^ s);
25  assign d_q = vq;
26  assign care_q = cq;
27 endmodule

```

چرا بیت اعتبار اختیاری نیست

نکته‌ای که در نگاه اول دیده نمی‌شود ولی مدار را از کار می‌اندازد این است: خانه‌ای که همه‌ی بیت‌های care آن صفر باشد، کاملاً X است و با هر کلمه‌ای منطبق می‌شود.

پس از رست، فلیپ‌فلاپ‌های care صفرند. بنابراین یک TCAM خالی — بدون بیت اعتبار — برای هر ورودی، شانزده انطباق گزارش می‌کند. بیت اعتبار در هر خانه دقیقاً جلوی همین را می‌گیرد و به‌عنوان یک ورودی جداگانه‌ی نوشتن هم بیرون داده شده است، تا بشود یک قاعده را بازنشسته کرد بدون اینکه محتوایش پاک شود.

$$\text{match}_i = \text{valid}_i \wedge (\bigwedge_k \text{agree}_{i,k})$$

این ویژگی در تست‌بنچ صریحاً بررسی شده است: هم اینکه TCAM خالی هرگز انطباق نمی‌دهد، و هم اینکه خانه‌ی تماماً X پس از معتبر شدن با هر کلمه‌ای منطبق می‌شود.

```

1  `include "tcam_pkg.vh"
2
3  module tcam_entry (
4      input wire          Clk,
5      input wire          RstN,
6
7      input wire          we,
8      input wire [ `TCAM_WIDTH-1:0] wdata,
9      input wire [ `TCAM_WIDTH-1:0] wmask,
10     input wire          wvalid,
11
12     input wire [ `TCAM_WIDTH-1:0] search,
13     output wire          match,
14
15     output wire [ `TCAM_WIDTH-1:0] q_value,
16     output wire [ `TCAM_WIDTH-1:0] q_mask,
17     output wire          q_valid
18 );
19     wire [ `TCAM_WIDTH-1:0] agree;
20     reg          valid;
21
22     genvar i;
23     generate
24         for (i = 0; i < `TCAM_WIDTH; i = i + 1) begin : g_cell
25             tcam_cell u_cell (
26                 .Clk(Clk), .RstN(RstN), .we(we),
27                 .d(wdata[i]), .care(wmask[i]), .s(search[i]),
28                 .agree(agree[i]), .d_q(q_value[i]), .care_q(q_mask[i])
29             );
30         end
31     endgenerate
32
33     always @(posedge Clk or negedge RstN) begin
34         if (!RstN) valid <= 1'b0;
35         else if (we) valid <= wvalid;
36     end
37
38     assign match = valid & (&agree);
39     assign q_valid = valid;
40 endmodule

```

مثال صورت آزمایش

صورت آزمایش می‌گوید اگر داده‌ی 01101110 در TCAM جست‌وجو شود، هر سه الگوی زیر انطباق دارند. رمزگذاری مقدار و ماسک هر کدام به این شکل درمی‌آید:

الگو	care	value	نتیجه برای 01101110
X1101XXX	01111000	01101000	انطباق
0X1X11X0	10101101	00101100	انطباق
0110XXXX	11110000	01100000	انطباق
1XXXXXXX	10000000	10000000	بدون انطباق
01101111	11111111	01101111	بدون انطباق

بررسی سطر اول به صورت دستی: $01101110 \oplus 01101000 = 00000110$ و $00000110 \wedge 01111000 = 00000000$ پس هیچ جایگاهی که قرار بوده مقایسه شود مخالفت نکرده است.

الگوهای صورت آزمایش ۸ بیتی‌اند و این TCAM ۱۶ بیتی است، پس بایت بالای هر الگو تماماً X گذاشته شده است. این یک دور زدن مسئله نیست؛ روش عادی ذخیره‌ی یک قاعده‌ی باریک در یک TCAM پهن‌تر همین است، و در تست‌بنچ با جست‌وجوی همان داده با دو بایت بالای متفاوت بررسی شده است.

چند انطباق هم‌زمان و اولویت

در مثال بالا یک کلمه‌ی جست‌وجو با سه قاعده منطبق شد. این حالت عادی است نه خطا — کل فایده‌ی TCAM همین است که قاعده‌های هم‌پوشان ذخیره شوند. اما چیزی باید انتخاب کند، و قرارداد تقریباً همه‌جایی این است که آدرس کوچک‌تر برنده است، تا قاعده‌های خاص‌تر جلوتر نوشته شوند.

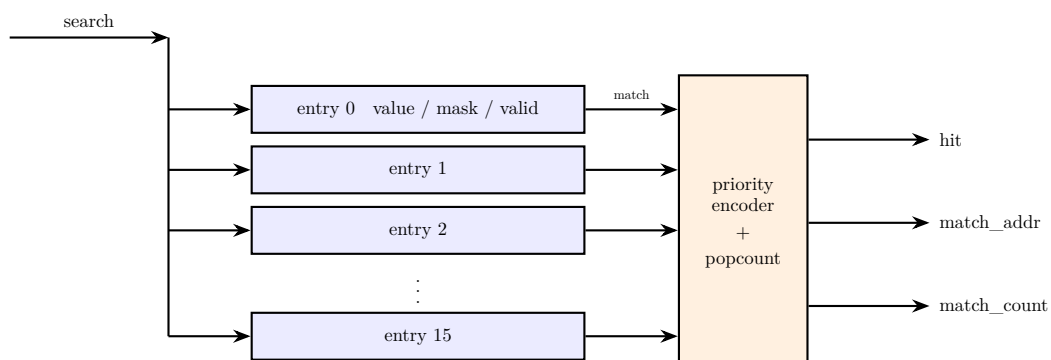
```

1 `include "tcam_pkg.vh"
2
3 module priority_encoder (
4     input wire [TCAM_ENTRIES-1:0] lines,
5     output reg [TCAM_AW-1:0] index,
6     output wire any
7 );
8     integer i;
9
10    assign any = |lines;
11
12    always @(*) begin
13        index = {TCAM_AW{1'b0}};
14        for (i = TCAM_ENTRIES-1; i >= 0; i = i - 1)
15            if (lines[i]) index = i[TCAM_AW-1:0];
16    end
17 endmodule

```

علاوه بر آدرس برنده، تعداد خانه‌های منطبق هم بیرون داده می‌شود. این خروجی تزئینی نیست: تفاوت میان «یک قاعده این کلمه را پوشش می‌دهد» و «چهار قاعده آن را پوشش می‌دهند» اطلاعات واقعی است، و از نظر تست تنها راهی است که می‌شود کدگذار اولویت درست را از کدگذاری که اتفاقی خانه‌ی درست را انتخاب کرده تشخیص داد.

پیمانه‌ی کامل



کلمه‌ی جست‌وجو به همه‌ی خانه‌ها هم‌زمان پخش می‌شود و مسیر از ورودی تا خطوط انطباق کاملاً ترکیبی است. حافظه فقط در محتواست، نه در جست‌وجو؛ و همین است که یک CAM را CAM می‌کند: یک جست‌وجو به اندازه‌ی یک زنجیره‌ی تأخیر گیت طول می‌کشد، نه شانزده خواندن از حافظه.

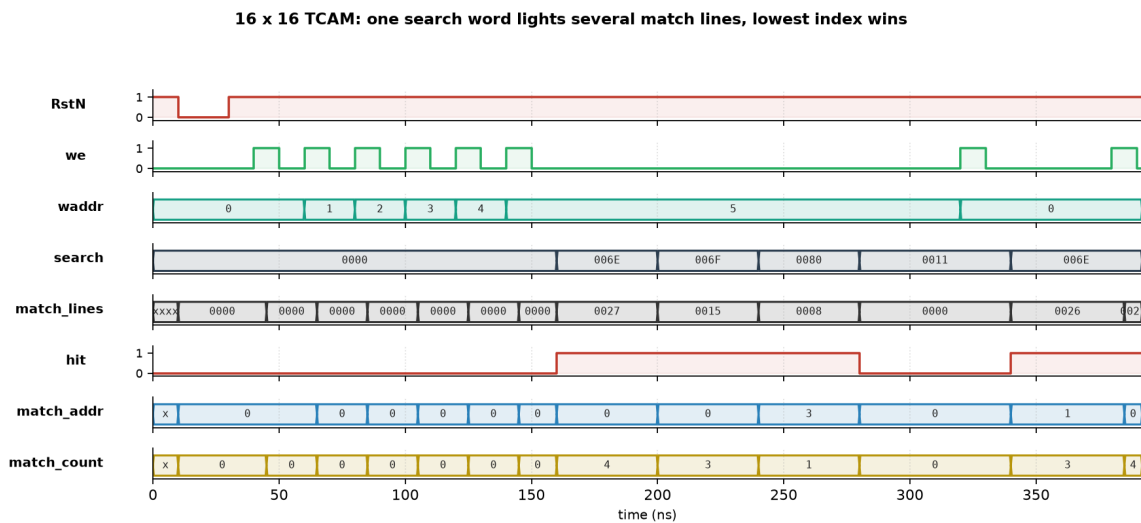
در پیاده‌سازی روی FPGA معمولاً خروجی‌ها را ثابت‌دار می‌کنند تا بستن زمان‌بندی ساده شود. اینجا این کار عمداً انجام نشده تا پیمانه دقیقاً همان جست‌وجوی شرکت‌پذیری باشد که صورت آزمایش توصیف کرده، بدون هیچ چیز اضاف‌ای در مسیر.

```

1  `include "tcam_pkg.vh"
2
3  module tcam (
4      input  wire          Clk,
5      input  wire          RstN,
6
7      input  wire          we,
8      input  wire [ `TCAM_AW-1:0] waddr,
9      input  wire [ `TCAM_WIDTH-1:0] wdata,
10     input  wire [ `TCAM_WIDTH-1:0] wmask,
11     input  wire          wvalid,
12
13     input  wire [ `TCAM_WIDTH-1:0] search,
14     output wire [ `TCAM_ENTRIES-1:0] match,
15     output wire          hit,
16     output wire [ `TCAM_AW-1:0] match_addr,
17     output wire [ `TCAM_CNTW-1:0] match_count,
18
19     // observation
20     input  wire [ `TCAM_AW-1:0] dbg_addr,
21     output wire [ `TCAM_WIDTH-1:0] dbg_value,
22     output wire [ `TCAM_WIDTH-1:0] dbg_mask,
23     output wire          dbg_valid
24 );
25 wire [ `TCAM_WIDTH-1:0] q_value [0: `TCAM_ENTRIES-1];
26 wire [ `TCAM_WIDTH-1:0] q_mask  [0: `TCAM_ENTRIES-1];
27 wire [ `TCAM_ENTRIES-1:0] q_valid;
28
29 genvar e;
30 generate
31     for (e = 0; e < `TCAM_ENTRIES; e = e + 1) begin : g_entry
32         tcam_entry u_entry (
33             .Clk(Clk), .RstN(RstN),
34             .we(we && (waddr == e[ `TCAM_AW-1:0])),
35             .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
36             .search(search), .match(match[e]),
37             .q_value(q_value[e]), .q_mask(q_mask[e]), .q_valid(q_valid[e])
38         );
39     end
40 endgenerate
41
42 priority_encoder u_pri (
43     .lines(match), .index(match_addr), .any(hit)
44 );
45
46 // Population count of the match lines.
47 integer k;
48 reg [ `TCAM_CNTW-1:0] cnt;
49 always @(*) begin
50     cnt = { `TCAM_CNTW{1'b0}};
51     for (k = 0; k < `TCAM_ENTRIES; k = k + 1)
52         cnt = cnt + match[k];
53 end
54 assign match_count = cnt;
55
56 assign dbg_value = q_value[dbg_addr];
57 assign dbg_mask  = q_mask [dbg_addr];
58 assign dbg_valid = q_valid[dbg_addr];
59 endmodule

```

شکل موج



شش الگوی جدول بالا در خانه‌های ۰ تا ۵ نوشته شده‌اند و سپس چند کلمه جست‌وجو می‌شود:

addr	count	خانه‌های منطبق	match_lines	جست‌وجو
۰	۴	۰, ۱, ۲, ۵	0x0027	0x006E
۰	۳	۰, ۲, ۴	0x0015	0x006F
۳	۱	۳	0x0008	0x0080
—	۰	—	0x0000	0x0011
پس از بازنشسته کردن خانه‌ی ۰:				
۱	۳	۱, ۲, ۵	0x0026	0x006E

سطر اول همان مثال صورت آزمایش است. سطر دوم جالب‌تر است: 01101111 فقط در آخرین بیت با داده‌ی قبلی فرق دارد، ولی مجموعه‌ی قاعده‌های منطبق کاملاً عوض می‌شود — 0X1X11X0 آن را رد می‌کند چون آخرین بیتش باید صفر باشد، و در عوض قاعده‌ی دقیق 01101111 حالا منطبق است.

دو سطر آخر رفتار اولویت را نشان می‌دهد: به محض بازنشسته شدن خانه‌ی ۰، خانه‌ی ۱ برنده می‌شود و شمارنده یکی کم می‌شود، بدون اینکه محتوای خانه‌ی ۰ پاک شده باشد.

توجه کنید که وقتی hit پایین است، مقدار match_addr بی‌معنی است. کدگذار اولویت در نبود هیچ انطباقی صفر بیرون می‌دهد و صفر یک آدرس معتبر است، پس مصرف‌کننده باید اول hit را ببیند. همین نکته باعث شد در تست‌بنچ، بررسی آدرس فقط زمانی انجام شود که مدل مرجع انتظار انطباق داشته باشد.

تست‌ها

چهار تست‌بنچ نوشته شده که از پایین به بالا مدار را می‌پوشانند.

(۱) سلول سه‌گانه — کامل. یک سلول فقط سه ورودی مؤثر دارد (مقدار ذخیره‌شده، بیت care و بیت جست‌وجو)، پس «کامل» اینجا واقعاً یعنی هر هشت ترکیب، در برابر جدول درستی‌ای که مستقیم از تعریف سه نماد $\{0, 1, X\}$ نوشته شده است.

```
1 `timescale 1ns/1ps
2
3 module tb_tcam_cell;
```

```

4  reg Clk = 1'b0, RstN = 1'b1, we = 1'b0, d = 1'b0, care = 1'b0, s = 1'b0;
5  wire agree, d_q, care_q;
6
7  tcam_cell dut (
8      .Clk(Clk), .RstN(RstN), .we(we), .d(d), .care(care), .s(s),
9      .agree(agree), .d_q(d_q), .care_q(care_q)
10 );
11
12 always #5 Clk = ~Clk;
13
14 integer pass = 0, fail = 0;
15 integer vi, ci, si;
16 reg expect_agree;
17
18 task automatic chk;
19     input [511:0] name;
20     input ok;
21     begin
22         if (ok) pass = pass + 1;
23         else begin
24             fail = fail + 1;
25             $display(" FAIL @%0t: %0s (d=%b care=%b s=%b agree=%b want=%b)",
26                 $time, name, d_q, care_q, s, agree, expect_agree);
27         end
28     end
29 endtask
30
31 task automatic store;
32     input v;
33     input c;
34     begin
35         @(negedge Clk); d = v; care = c; we = 1'b1;
36         @(negedge Clk); we = 1'b0;
37     end
38 endtask
39
40 initial begin
41     @(negedge Clk); RstN = 1'b0;
42     repeat (2) @(negedge Clk); RstN = 1'b1;
43     @(negedge Clk);
44
45     chk("reset stores X (care = 0)", care_q === 1'b0);
46     s = 1'b0; #1; chk("reset cell agrees with 0", agree === 1'b1);
47     s = 1'b1; #1; chk("reset cell agrees with 1", agree === 1'b1);
48
49     // ---- all eight combinations ----
50     for (vi = 0; vi < 2; vi = vi + 1)
51         for (ci = 0; ci < 2; ci = ci + 1) begin
52             store(vi[0], ci[0]);
53             chk("stored value read back", d_q === vi[0]);
54             chk("stored care read back", care_q === ci[0]);
55             for (si = 0; si < 2; si = si + 1) begin
56                 s = si[0]; #1;
57                 // X agrees with everything; 0 and 1 agree only with themselves.
58                 expect_agree = (ci[0] == 1'b0) ? 1'b1 : (vi[0] == si[0]);
59                 chk("truth table", agree === expect_agree);
60             end
61         end
62
63     // ---- a cell is only rewritten when we is high ----
64     store(1'b1, 1'b1);
65     @(negedge Clk); d = 1'b0; care = 1'b1; we = 1'b0;
66     @(negedge Clk);
67     chk("no write without we", d_q === 1'b1 && care_q === 1'b1);
68
69     $display("tb_tcam_cell: Passed: %0d, Failed: %0d", pass, fail);
70     if (fail != 0) $fatal(1);
71     $finish;
72 end
73 endmodule

```

۲) یک خانه — جاروب کامل فضای جست‌وجو. یک خانه‌ی ۱۶ بیتی فقط ۶۵۵۳۶ کلمه‌ی جست‌وجوی ممکن دارد، پس برای هر الگوی ذخیره‌شده می‌شود همه‌ی آن‌ها را بررسی کرد؛ نه نمونه‌برداری و نه زیرمجموعه‌ی تصادفی. علاوه بر خود خط انطباق، تعداد کلمه‌های منطبق هم شمرده و با مقدار نظری مقایسه می‌شود. الگویی با n بیت X باید دقیقاً با 2^n کلمه منطبق شود:

الگو	بیت‌های X	تعداد کلمه‌های منطبق
mask = 0xFFFF	۰	۱
mask = 0xFF00	۸	۲۵۶
mask = 0x0F0F	۸	۲۵۶
mask = 0x8001	۱۴	۱۶۳۸۴
mask = 0x0000	۱۶	۶۵۵۳۶

این بررسی چیزی می‌گیرد که مقایسه‌ی تک‌تک نمی‌گیرد: اگر یک بیت care به سلول اشتباهی وصل شده باشد، برای بیشتر کلمه‌ها پاسخ همچنان درست درمی‌آید ولی اندازه مجموعه‌ی منطبق عوض می‌شود.

```

1 `timescale 1ns/1ps
2 `include "tcam_pkg.vh"
3
4 module tb_tcam_entry;
5     reg                Clk = 1'b0, RstN = 1'b1;
6     reg                we = 1'b0, wvalid = 1'b1;
7     reg [`TCAM_WIDTH-1:0] wdata = 16'd0, wmask = 16'd0;
8     reg [`TCAM_WIDTH-1:0] search = 16'd0;
9
10    wire                match;
11    wire [`TCAM_WIDTH-1:0] q_value, q_mask;
12    wire                q_valid;
13
14    tcam_entry dut (
15        .Clk(Clk), .RstN(RstN),
16        .we(we), .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
17        .search(search), .match(match),
18        .q_value(q_value), .q_mask(q_mask), .q_valid(q_valid)
19    );
20
21    always #5 Clk = ~Clk;
22
23    integer pass = 0, fail = 0;
24    integer p, w, hits;
25    reg expect_match;
26
27    task automatic chk;
28        input [511:0] name;
29        input ok;
30        begin
31            if (ok) pass = pass + 1;
32            else begin
33                fail = fail + 1;
34                $display(" FAIL @%0t: %0s (value=%h mask=%h search=%h match=%b want=%b)",
35                    $time, name, q_value, q_mask, search, match, expect_match);
36            end
37        end
38    endtask
39
40    task automatic store;
41        input [`TCAM_WIDTH-1:0] v;
42        input [`TCAM_WIDTH-1:0] m;
43        input vld;
44        begin
45            @(negedge Clk); wdata = v; wmask = m; wvalid = vld; we = 1'b1;
46            @(negedge Clk); we = 1'b0;
47            #1;
48        end
49    endtask
50
51    task automatic sweep;
52        input [`TCAM_WIDTH-1:0] v;
53        input [`TCAM_WIDTH-1:0] m;
54        input integer want_hits;
55        begin
56            store(v, m, 1'b1);
57            hits = 0;
58            for (w = 0; w < 65536; w = w + 1) begin
59                search = w[`TCAM_WIDTH-1:0];
60                #1;
61                expect_match = (((w[`TCAM_WIDTH-1:0] ^ v) & m) == {`TCAM_WIDTH{1'b0}});
62                if (match != expect_match) begin
63                    fail = fail + 1;
64                    $display(" FAIL @%0t: sweep value=%h mask=%h search=%h match=%b want=%b",
65                        $time, v, m, search, match, expect_match);
66                end else pass = pass + 1;
67                if (match) hits = hits + 1;
68            end
69        end

```

```

69         if (want_hits >= 0) begin
70             expect_match = 1'bx;
71             chk("number of matching words over the whole space",
72                 hits == want_hits);
73         end
74     end
75 endtask
76
77 initial begin
78     @(negedge Clk); RstN = 1'b0;
79     repeat (2) @(negedge Clk); RstN = 1'b1;
80     @(negedge Clk); #1;
81
82     chk("entry invalid after reset", q_valid === 1'b0);
83     search = 16'hA5A5; #1; chk("invalid entry does not match", match === 1'b0);
84     search = 16'h0000; #1; chk("invalid entry does not match zero", match === 1'b0);
85
86     sweep(16'h1234, 16'hFFFF, 1);
87     sweep(16'h0000, 16'h0000, 65536);
88     sweep(16'hAB00, 16'hFF00, 256);
89     sweep(16'h5A5A, 16'h0F0F, 256);
90     sweep(16'hFFFF, 16'h8001, 16384);
91     store(16'h1234, 16'hFFFF, 1'b1);
92     search = 16'h1234; #1;
93     chk("stored word matches", match === 1'b1);
94     store(16'h1234, 16'hFFFF, 1'b0);
95     search = 16'h1234; #1;
96     chk("retired entry stops matching", match === 1'b0);
97     chk("contents survive retirement", q_value === 16'h1234 && q_mask === 16'hFFFF);
98
99     $display("tb_tcam_entry: Passed: %0d, Failed: %0d", pass, fail);
100    if (fail != 0) $fatal(1);
101    $finish;
102 end
103 endmodule

```

۳) پیمانه‌ی کامل در برابر مدل مرجع. مدل چند خط رفتاری بدون سلول، بدون کدگذار اولویت و بدون درخت شمارش است، پس نمی‌تواند اشتباه ساختاری طراحی را تکرار کند.

مهم‌تر از خود مدل، نحوه‌ی راندن آن است: هم محتوای TCAM و هم کلمه‌های جست‌وجو تصادفی‌اند، و ماسک‌ها عمداً به سمت داشتن X زیاد سوگیری شده‌اند (\$random & \$random). دلایل این است که رفتار جالب TCAM وقتی ظاهر می‌شود که قاعده‌ها هم‌پوشانی داشته باشند، و مجموعه‌ی بردارهای دست‌نویس دقیقاً از هم‌پوشانی پرهیز می‌کند. در پایان، یک پیکربندی هم روی تمام ۶۵۵۳۶ کلمه‌ی ممکن جاروب می‌شود تا دست‌کم یک نقشه‌ی کامل از فضای انطباق بررسی شده باشد، نه نمونه‌برداری شده.

```

1  `timescale 1ns/1ps
2  `include "tcam_pkg.vh"
3
4  module tb_tcam;
5      reg                Clk = 1'b0, RstN = 1'b1;
6      reg                we = 1'b0, wvalid = 1'b1;
7      reg [`TCAM_AW-1:0] waddr = 4'd0;
8      reg [`TCAM_WIDTH-1:0] wdata = 16'd0, wmask = 16'd0;
9      reg [`TCAM_WIDTH-1:0] search = 16'd0;
10     reg [`TCAM_AW-1:0] dbg_addr = 4'd0;
11
12     wire [`TCAM_ENTRIES-1:0] match;
13     wire                hit;
14     wire [`TCAM_AW-1:0] match_addr;
15     wire [`TCAM_CNTW-1:0] match_count;
16     wire [`TCAM_WIDTH-1:0] dbg_value, dbg_mask;
17     wire                dbg_valid;
18
19     tcam dut (
20         .Clk(Clk), .RstN(RstN),
21         .we(we), .waddr(waddr), .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
22         .search(search), .match(match), .hit(hit),
23         .match_addr(match_addr), .match_count(match_count),
24         .dbg_addr(dbg_addr), .dbg_value(dbg_value), .dbg_mask(dbg_mask),
25         .dbg_valid(dbg_valid)
26     );
27
28     always #5 Clk = ~Clk;
29
30     integer pass = 0, fail = 0;
31     integer c, e, w, seed;
32
33     reg [`TCAM_WIDTH-1:0] m_val [0:`TCAM_ENTRIES-1];
34     reg [`TCAM_WIDTH-1:0] m_mask [0:`TCAM_ENTRIES-1];
35     reg [`TCAM_ENTRIES-1:0] m_vld;

```



```

36
37 reg [`TCAM_ENTRIES-1:0] exp_match;
38 reg [`TCAM_AW-1:0] exp_addr;
39 reg [`TCAM_CNTW-1:0] exp_count;
40 reg exp_hit;
41
42 task automatic chk;
43     input [511:0] name;
44     input ok;
45     begin
46         if (ok) pass = pass + 1;
47         else begin
48             fail = fail + 1;
49             $display(" FAIL @%0t: %0s (search=%h match=%h/%h hit=%b/%b addr=%0d/%0d cnt=%0d/%0d)",
50                 $time, name, search, match, exp_match, hit, exp_hit,
51                 match_addr, exp_addr, match_count, exp_count);
52         end
53     end
54 endtask
55
56 task automatic store;
57     input [`TCAM_AW-1:0] a;
58     input [`TCAM_WIDTH-1:0] v;
59     input [`TCAM_WIDTH-1:0] m;
60     input vld;
61     begin
62         @(negedge Clk);
63         waddr = a; wdata = v; wmask = m; wvalid = vld; we = 1'b1;
64         @(negedge Clk); we = 1'b0;
65         m_val[a] = v;
66         m_mask[a] = m;
67         m_vld[a] = vld;
68         #1;
69     end
70 endtask
71
72 task automatic model;
73     integer g;
74     begin
75         exp_match = {`TCAM_ENTRIES{1'b0}};
76         exp_count = {`TCAM_CNTW{1'b0}};
77         for (g = 0; g < `TCAM_ENTRIES; g = g + 1)
78             if (m_vld[g] && (((search ^ m_val[g]) & m_mask[g]) == {`TCAM_WIDTH{1'b0}})) begin
79                 exp_match[g] = 1'b1;
80                 exp_count = exp_count + 1'b1;
81             end
82         exp_hit = |exp_match;
83         exp_addr = {`TCAM_AW{1'b0}};
84         for (g = `TCAM_ENTRIES-1; g >= 0; g = g - 1)
85             if (exp_match[g]) exp_addr = g[`TCAM_AW-1:0];
86     end
87 endtask
88
89 task automatic probe;
90     input [`TCAM_WIDTH-1:0] s;
91     begin
92         search = s; #1;
93         model;
94         chk("match lines", match == exp_match);
95         chk("hit", hit == exp_hit);
96         chk("count", match_count == exp_count);
97         if (exp_hit) chk("priority address", match_addr == exp_addr);
98     end
99 endtask
100
101 initial begin
102     @(negedge Clk); RstN = 1'b0;
103     repeat (2) @(negedge Clk); RstN = 1'b1;
104     @(negedge Clk);
105
106     for (e = 0; e < `TCAM_ENTRIES; e = e + 1) begin
107         m_val[e] = 16'd0;
108         m_mask[e] = 16'd0;
109         m_vld[e] = 1'b0;
110     end
111     #1;
112
113     probe(16'h0000);
114     probe(16'hFFFF);
115     probe(16'h5A5A);
116     chk("empty TCAM never hits", hit == 1'b0 && match_count == 5'd0);
117
118     store(4'd7, 16'h1234, 16'hFF0F, 1'b1);
119     dbg_addr = 4'd7; #1;
120     chk("value stored", dbg_value == 16'h1234);
121     chk("mask stored", dbg_mask == 16'hFF0F);
122     chk("valid stored", dbg_valid == 1'b1);

```

```

123     dbg_addr = 4'd6; #1;
124     chk("neighbouring entry untouched", dbg_valid === 1'b0);
125
126     seed = 32'd90909;
127     for (c = 0; c < 60; c = c + 1) begin
128         for (e = 0; e < `TCAM_ENTRIES; e = e + 1) begin
129             store(e[`TCAM_AW-1:0],
130                 $random(seed),
131                 {$random(seed)} & {$random(seed)},
132                 (($random(seed)) % 8) != 0));
133         end
134         for (w = 0; w < 200; w = w + 1)
135             probe($random(seed));
136     end
137
138     for (e = 0; e < `TCAM_ENTRIES; e = e + 1)
139         store(e[`TCAM_AW-1:0],
140             $random(seed),
141             {$random(seed)} & {$random(seed)},
142             1'b1);
143     for (w = 0; w < 65536; w = w + 1)
144         probe(w[`TCAM_WIDTH-1:0]);
145
146     $display("tb_tcam: Passed: %0d, Failed: %0d", pass, fail);
147     if (fail != 0) $fatal(1);
148     $finish;
149 end
150 endmodule

```

۴) انطباق مستقیم با صورت آزمایش. در این تست‌بنچ الگوها به صورت متن و با همان نماد $\{0, 1, X\}$ صورت آزمایش نوشته می‌شوند و خود تست‌بنچ آن‌ها را به زوج مقدار/ماسک تبدیل می‌کند:

```

store_pattern(4'd0, "XXXXXXXX1101XXX");
store_pattern(4'd1, "XXXXXXXX0X1X1X0");
store_pattern(4'd2, "XXXXXXXX0110XXXX");

```

این کار دو فایده دارد. اول اینکه بررسی‌ها کنار صورت آزمایش خوانا می‌مانند. دوم و مهم‌تر اینکه خود رمزگذاری مقدار/ماسک بخشی از چیزی می‌شود که آزموده می‌شود: اگر رمزگذاری غلط بود، مثال کار شده‌ی صورت آزمایش از انطباق می‌افتاد.

```

1  `timescale 1ns/1ps
2  `include "tcam_pkg.vh"
3
4  module tb_tcam_spec;
5      reg          Clk = 1'b0, RstN = 1'b1;
6      reg          we = 1'b0, wvalid = 1'b1;
7      reg [`TCAM_AW-1:0] waddr = 4'd0;
8      reg [`TCAM_WIDTH-1:0] wdata = 16'd0, wmask = 16'd0;
9      reg [`TCAM_WIDTH-1:0] search = 16'd0;
10     reg [`TCAM_AW-1:0] dbg_addr = 4'd0;
11
12     wire [`TCAM_ENTRIES-1:0] match;
13     wire          hit;
14     wire [`TCAM_AW-1:0] match_addr;
15     wire [`TCAM_CNTW-1:0] match_count;
16     wire [`TCAM_WIDTH-1:0] dbg_value, dbg_mask;
17     wire          dbg_valid;
18
19     tcam dut (
20         .Clk(Clk), .RstN(RstN),
21         .we(we), .waddr(waddr), .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
22         .search(search), .match(match), .hit(hit),
23         .match_addr(match_addr), .match_count(match_count),
24         .dbg_addr(dbg_addr), .dbg_value(dbg_value), .dbg_mask(dbg_mask),
25         .dbg_valid(dbg_valid)
26     );
27
28     always #5 Clk = ~Clk;
29
30     integer pass = 0, fail = 0;
31
32     task automatic chk;
33         input [511:0] name;
34         input          ok;
35         begin
36             if (ok) pass = pass + 1;
37             else begin
38                 fail = fail + 1;
39                 $display(" FAIL @%0t: %0s (search=%h match=%h hit=%b addr=%0d cnt=%0d)",
40                     $time, name, search, match, hit, match_addr, match_count);
41             end
42         end
43     endtask

```

```

42     end
43 endtask
44
45 task automatic store;
46     input [`TCAM_AW-1:0] a;
47     input [`TCAM_WIDTH-1:0] v;
48     input [`TCAM_WIDTH-1:0] m;
49     input vld;
50     begin
51         @(negedge Clk);
52         waddr = a; wdata = v; wmask = m; wvalid = vld; we = 1'b1;
53         @(negedge Clk); we = 1'b0;
54         #1;
55     end
56 endtask
57
58 task automatic store_pattern;
59     input [`TCAM_AW-1:0] a;
60     input [8*`TCAM_WIDTH-1:0] pat;
61     reg [`TCAM_WIDTH-1:0] v, m;
62     reg [7:0] ch;
63     integer i;
64     begin
65         v = `{`TCAM_WIDTH{1'b0}};
66         m = `{`TCAM_WIDTH{1'b0}};
67         for (i = 0; i < `TCAM_WIDTH; i = i + 1) begin
68             ch = pat[8*(`TCAM_WIDTH-1-i) +: 8];
69             if (ch == "1") begin
70                 v[`TCAM_WIDTH-1-i] = 1'b1;
71                 m[`TCAM_WIDTH-1-i] = 1'b1;
72             end else if (ch == "0") begin
73                 m[`TCAM_WIDTH-1-i] = 1'b1;
74             end
75         end
76         store(a, v, m, 1'b1);
77     end
78 endtask
79
80 task automatic look;
81     input [`TCAM_WIDTH-1:0] s;
82     begin
83         search = s; #1;
84     end
85 endtask
86
87 initial begin
88     @(negedge Clk); RstN = 1'b0;
89     repeat (2) @(negedge Clk); RstN = 1'b1;
90     @(negedge Clk); #1;
91
92     store_pattern(4'd0, "XXXXXXXXX1101XXX");
93     store_pattern(4'd1, "XXXXXXXXX0X1X11X0");
94     store_pattern(4'd2, "XXXXXXXXX0110XXXX");
95     store_pattern(4'd3, "XXXXXXXXX1XXXXXXX");
96     store_pattern(4'd4, "XXXXXXXXX01101111");
97     store_pattern(4'd5, "XXXXXXXXX01101110");
98
99     dbg_addr = 4'd0; #1;
100     chk("pattern X1101XXX encodes to mask 0x0078", dbg_mask === 16'h0078);
101     chk("pattern X1101XXX encodes to value 0x0068", dbg_value === 16'h0068);
102
103     look(16'h006E); // the data word 01101110
104     chk("sheet: X1101XXX matches", match[0] === 1'b1);
105     chk("sheet: 0X1X11X0 matches", match[1] === 1'b1);
106     chk("sheet: 0110XXXX matches", match[2] === 1'b1);
107     chk("1XXXXXXX does not match", match[3] === 1'b0);
108     chk("01101111 does not match", match[4] === 1'b0);
109     chk("01101110 matches exactly", match[5] === 1'b1);
110     chk("four entries matched in total", match_count === 5'd4);
111     chk("there was a hit", hit === 1'b1);
112     chk("lowest matching index wins", match_addr === 4'd0);
113
114     look(16'hFF6E);
115     chk("high byte is don't-care: same match lines",
116         match[5:0] === 6'b100111 && match_count === 5'd4);
117     look(16'h5A6E);
118     chk("high byte is don't-care: any value works", match_count === 5'd4);
119
120     look(16'h006F);
121     chk("X1101XXX still matches 01101111", match[0] === 1'b1);
122     chk("0X1X11X0 rejects it (last bit must be 0)", match[1] === 1'b0);
123     chk("0110XXXX still matches", match[2] === 1'b1);
124     chk("01101111 now matches exactly", match[4] === 1'b1);
125     chk("01101110 no longer matches", match[5] === 1'b0);
126     chk("three entries matched", match_count === 5'd3);
127
128     look(16'h006E);

```

```

129     chk("winner is entry 0", match_addr === 4'd0);
130     store(4'd0, 16'h0068, 16'h0078, 1'b0); // retire entry 0
131     look(16'h006E);
132     chk("after retiring entry 0 the winner is entry 1", match_addr === 4'd1);
133     chk("and one fewer entry matches", match_count === 5'd3);
134     store(4'd0, 16'h0068, 16'h0078, 1'b1); // bring it back
135     look(16'h006E);
136     chk("re-validating restores the winner", match_addr === 4'd0);
137     chk("and the match count", match_count === 5'd4);
138
139     store_pattern(4'd15, "XXXXXXXXXXXXXX"); // all X
140     look(16'h0000); chk("all-X entry matches 0x0000", match[15] === 1'b1);
141     look(16'hFFFF); chk("all-X entry matches 0xFFFF", match[15] === 1'b1);
142     look(16'hA5A5); chk("all-X entry matches anything", match[15] === 1'b1);
143
144     store_pattern(4'd14, "0000111100001111"); // fully specified
145     look(16'h0F0F); chk("exact entry matches its word", match[14] === 1'b1);
146     look(16'h0F0E); chk("exact entry rejects a neighbour", match[14] === 1'b0);
147
148     store(4'd15, 16'h0000, 16'h0000, 1'b0);
149     look(16'hA5A5);
150     chk("retired all-X entry is silent", match[15] === 1'b0);
151
152     @(negedge Clk); RstN = 1'b0;
153     repeat (2) @(negedge Clk); RstN = 1'b1;
154     @(negedge Clk);
155     store_pattern(4'd0, "1111111111111111");
156     look(16'h0000);
157     chk("no match means no hit", hit === 1'b0 && match_count === 5'd0);
158     chk("match lines are all clear", match === 16'h0000);
159
160     $display("tb_tcam_spec: Passed: %0d, Failed: %0d", pass, fail);
161     if (fail != 0) $fatal(1);
162     $finish;
163 end
164 endmodule

```

نتایج تست

```

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_tcam_cell ===
tb_tcam_cell: Passed: 20, Failed: 0
PASS: tb_tcam_cell

=== tb_tcam_entry ===
tb_tcam_entry: Passed: 327691, Failed: 0
PASS: tb_tcam_entry

=== tb_tcam ===
tb_tcam: Passed: 300532, Failed: 0
PASS: tb_tcam

=== tb_tcam_spec ===
tb_tcam_spec: Passed: 32, Failed: 0
PASS: tb_tcam_spec

summary: 4/4 passed, 0 failed

```

خروجی سنتز (Quartus Prime)

طرح روی Intel Quartus Prime Lite Edition 21.1.0 با موجودیت سطح بالای tcam سنتز و Route & Place شد. دستگاه هدف: Cyclone V (5CGXFC7C7F23C8).

مورد	نتیجه
وضعیت کامپایل	موفق (۰ خطا، ۱۳ هشدار)
Logic utilization (ALMs)	۳۵۳/۵۶۴۸۰ (< ۱٪)
Total registers	۵۲۸
Total pins	۱۱۹/۲۶۸ (۴۴٪)
Block memory bits	•
DSP Blocks	•

